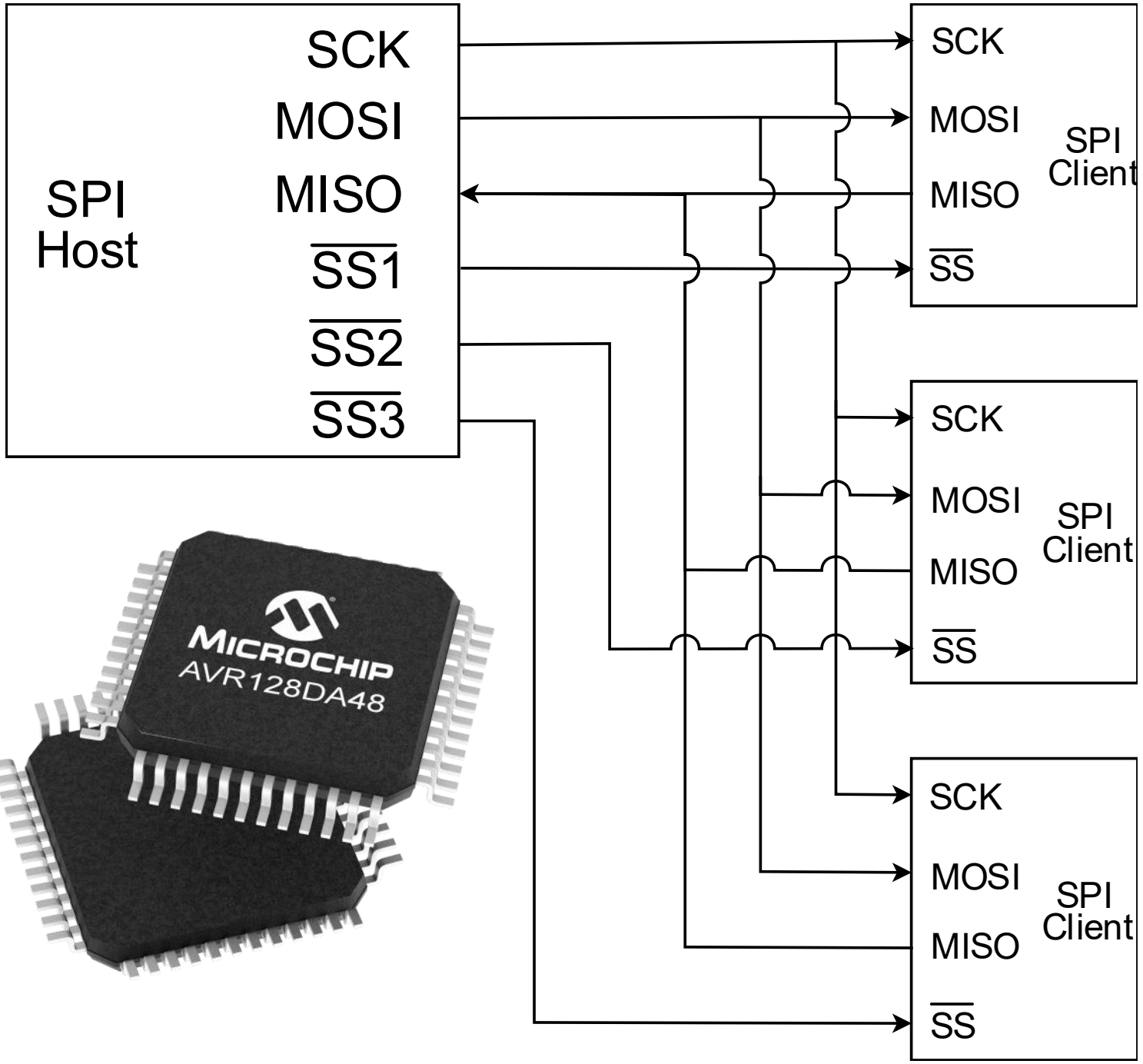
“Bare-Metal Embedded Systems with AVR128DA48 Course”

Day 13 - SPI Master Mode



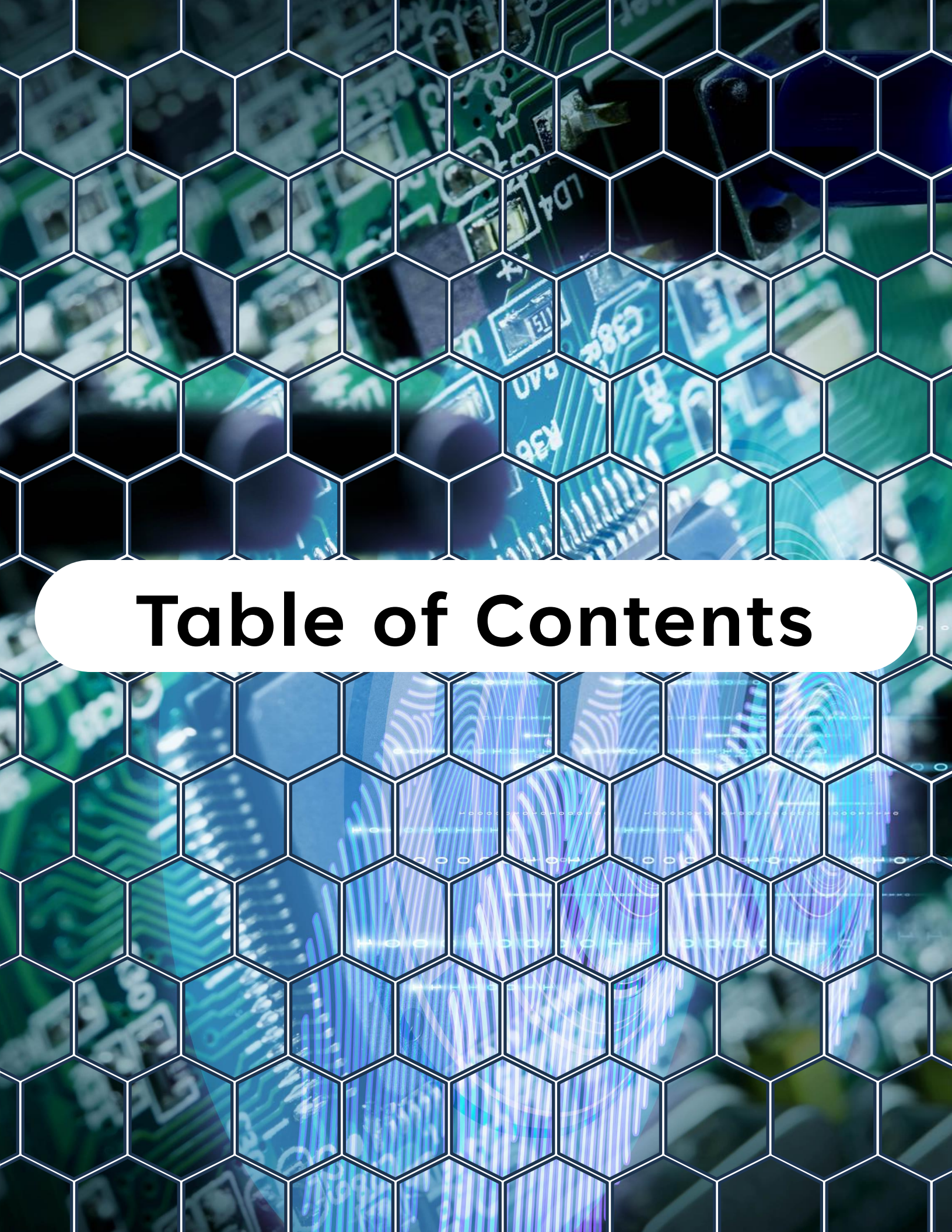
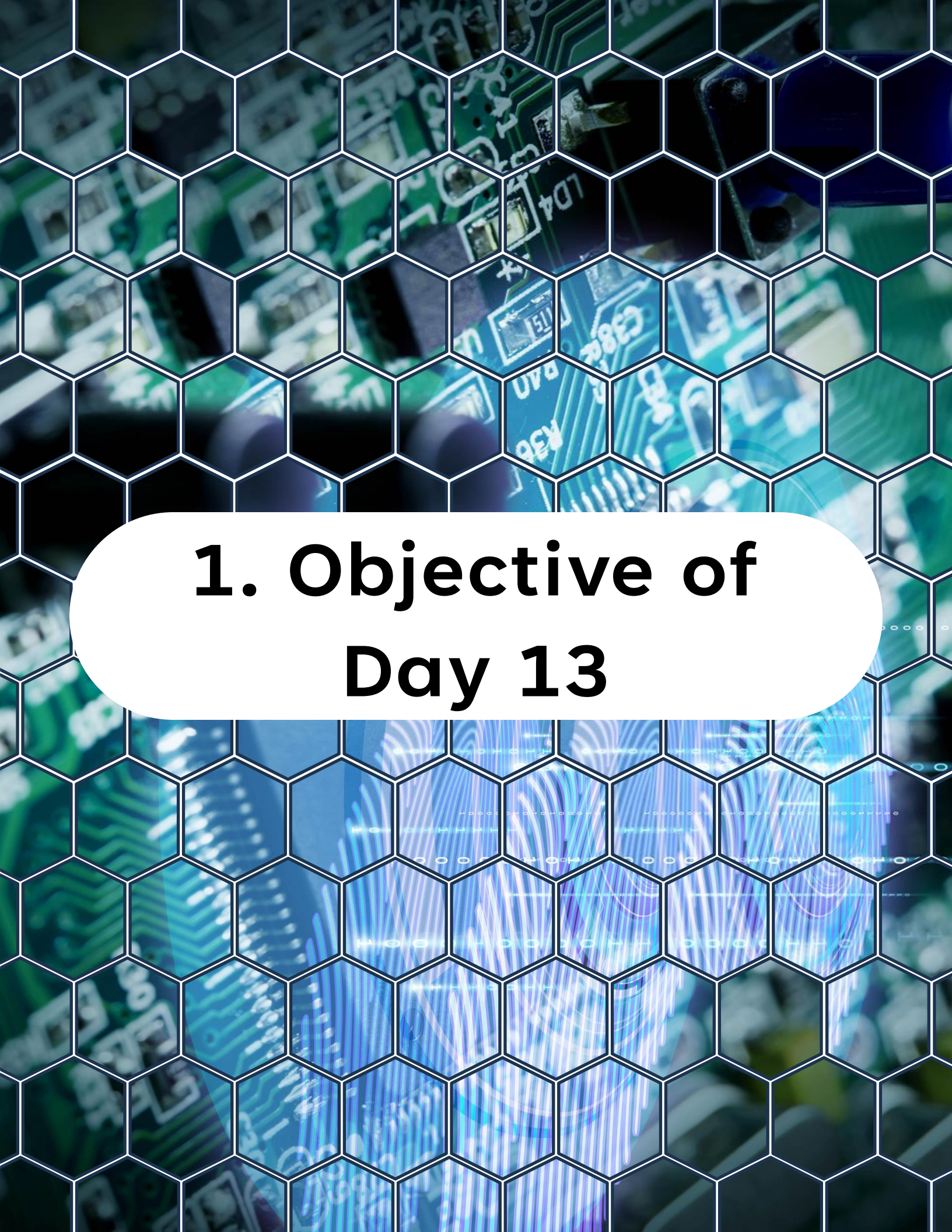


Table of Contents

Table of Contents



1. Objective of Day 13

1. Objective of Day 13

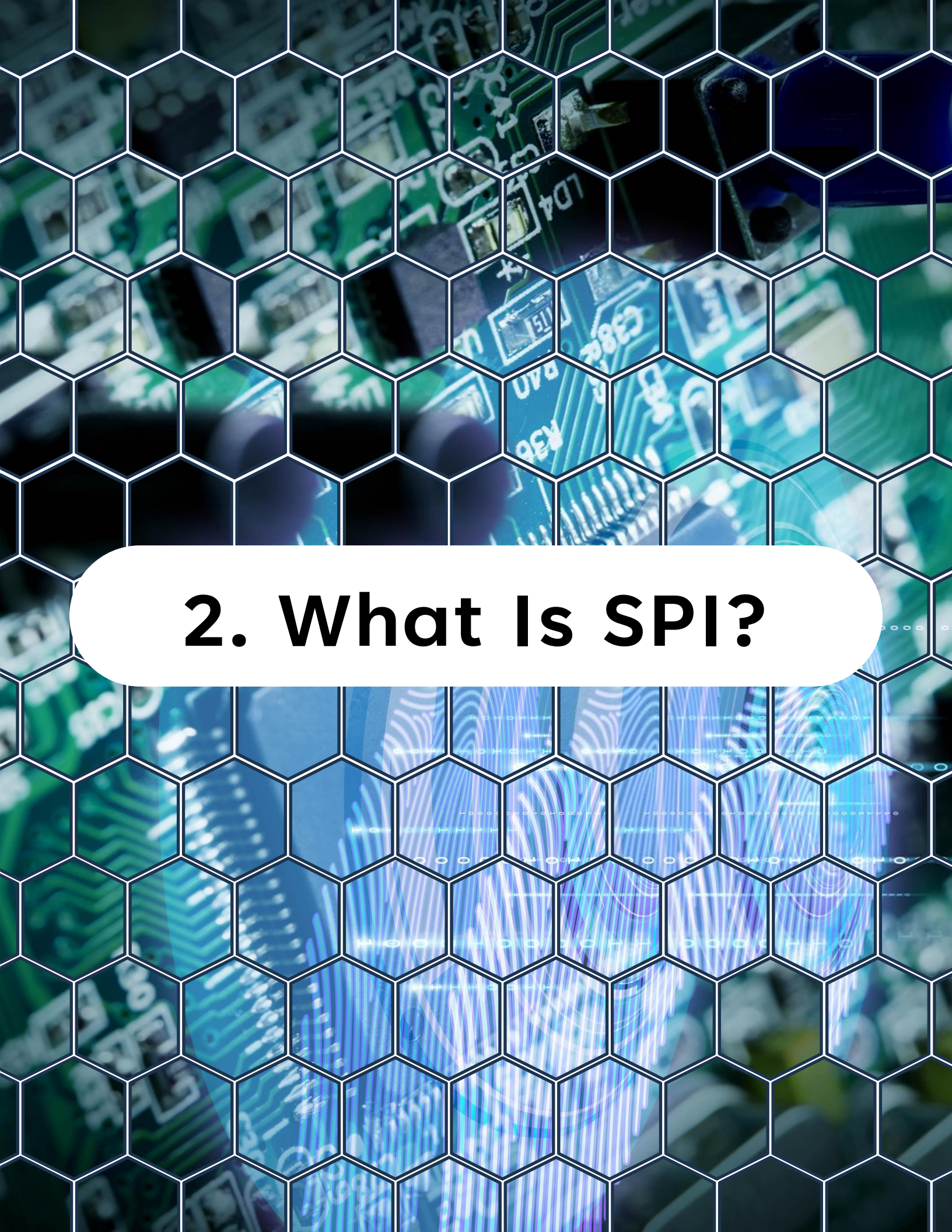
Today your MCU starts talking to other hardware devices.

You will configure the **SPI peripheral in Master mode** and perform real full-duplex transfers at register level.

By the end of this day, you will:

- Understand SPI signaling (**MOSI**, **MISO**, **SCK**, **SS**)
- Configure SPI in Master mode
- Select clock polarity and phase correctly
- Perform blocking SPI transfers
- Control chip select manually
- Debug common SPI failures

SPI is fundamental in embedded systems. Sensors, ADCs, DACs, displays, flash memories, RF chips, and many other devices rely on it.



2. What Is SPI?

2. What Is SPI?

SPI stands for Serial Peripheral Interface.

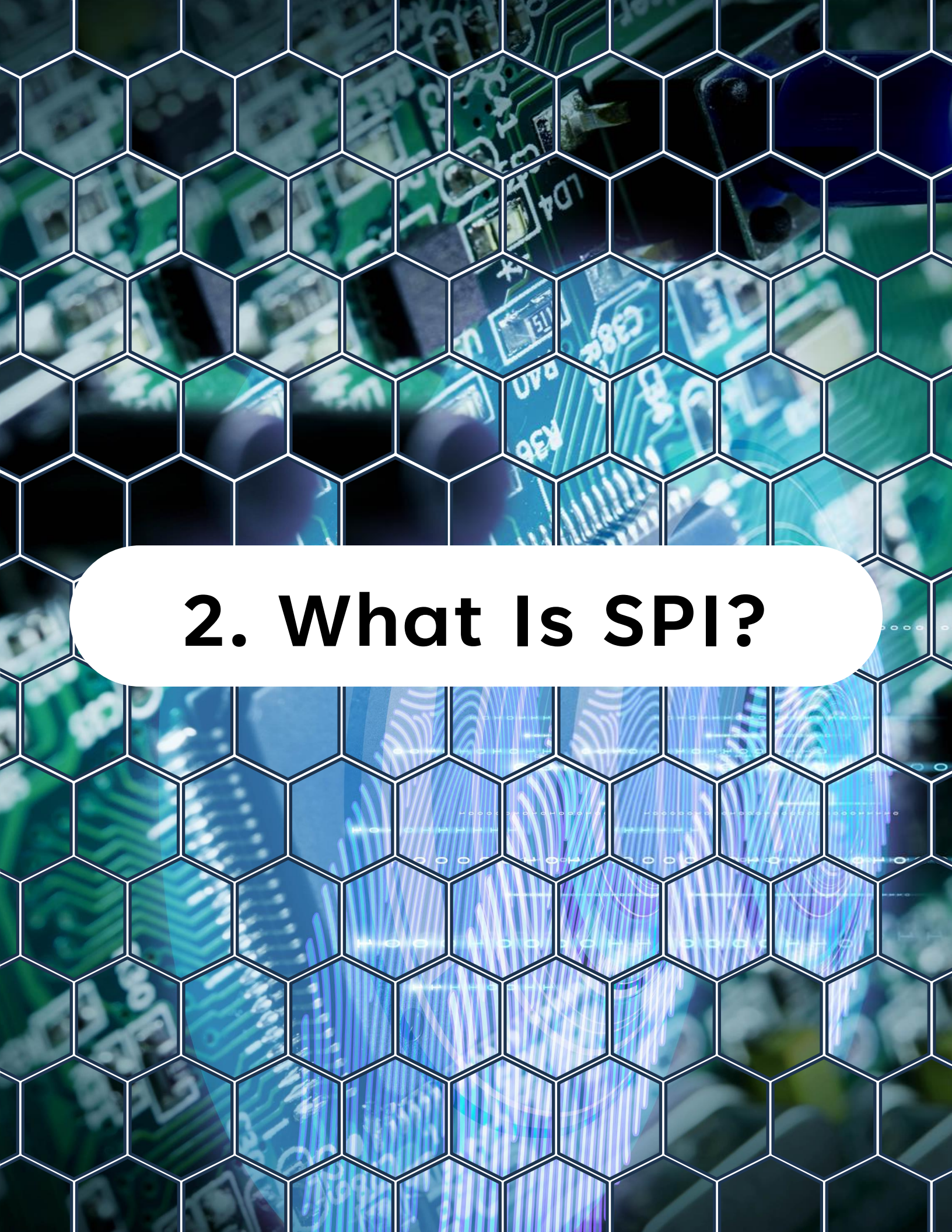
It is:

- Synchronous
- Full-duplex
- Master-slave architecture
- Clock-driven

Basic signals:

- **MOSI**: Master Out Slave In
- **MISO**: Master In Slave Out
- **SCK**: Serial Clock
- **SS** (CS): Slave Select (chip select)

Unlike UART, SPI has a clock line. That makes it faster and more deterministic.



2. What Is SPI?

2. What Is SPI?

SPI stands for Serial Peripheral Interface.

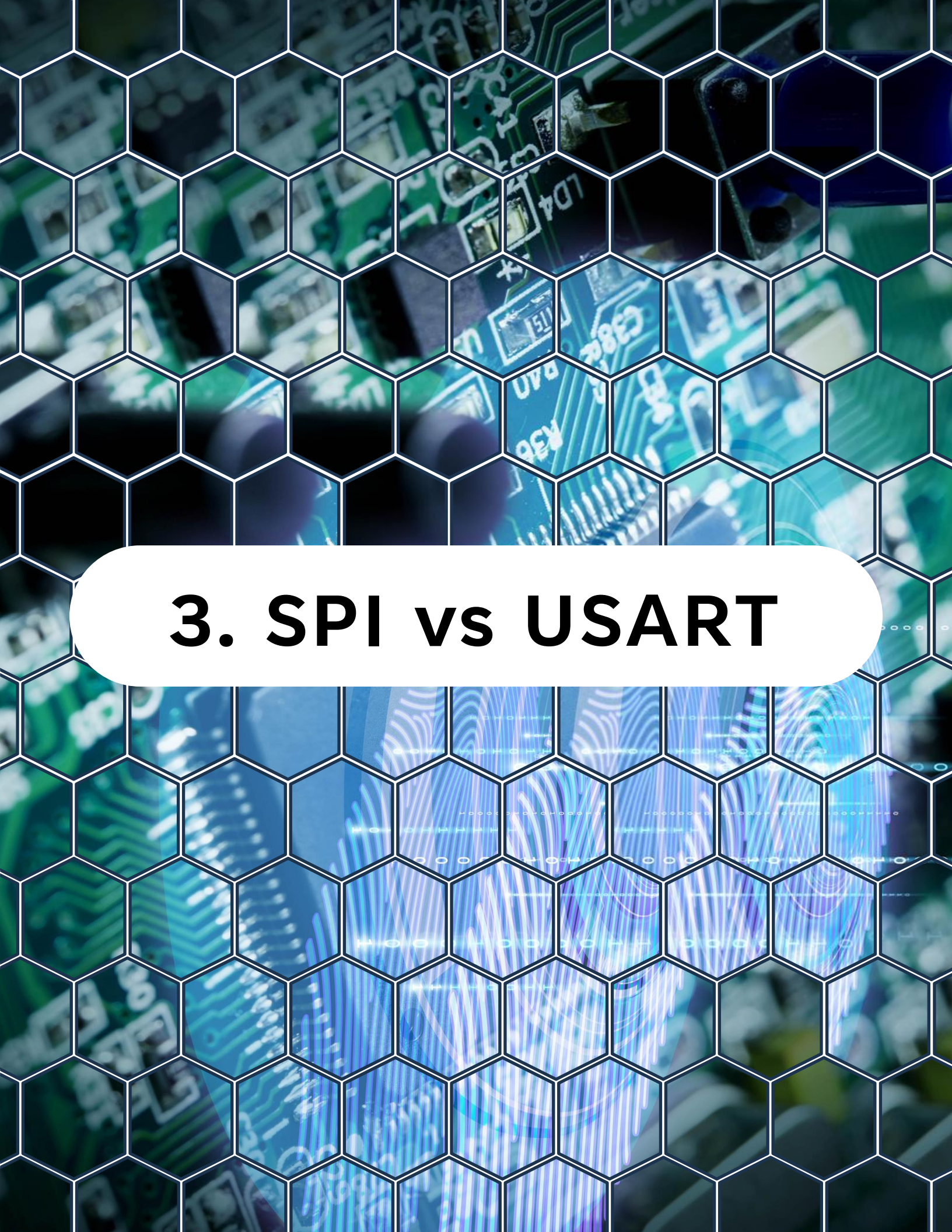
It is:

- Synchronous
- Full-duplex
- Master-slave architecture
- Clock-driven

Basic signals:

- **MOSI**: Master Out Slave In
- **MISO**: Master In Slave Out
- **SCK**: Serial Clock
- **SS** (CS): Slave Select (chip select)

Unlike UART, SPI has a clock line. That makes it faster and more deterministic.



3. SPI vs USART

3. SPI vs USART

SPI:

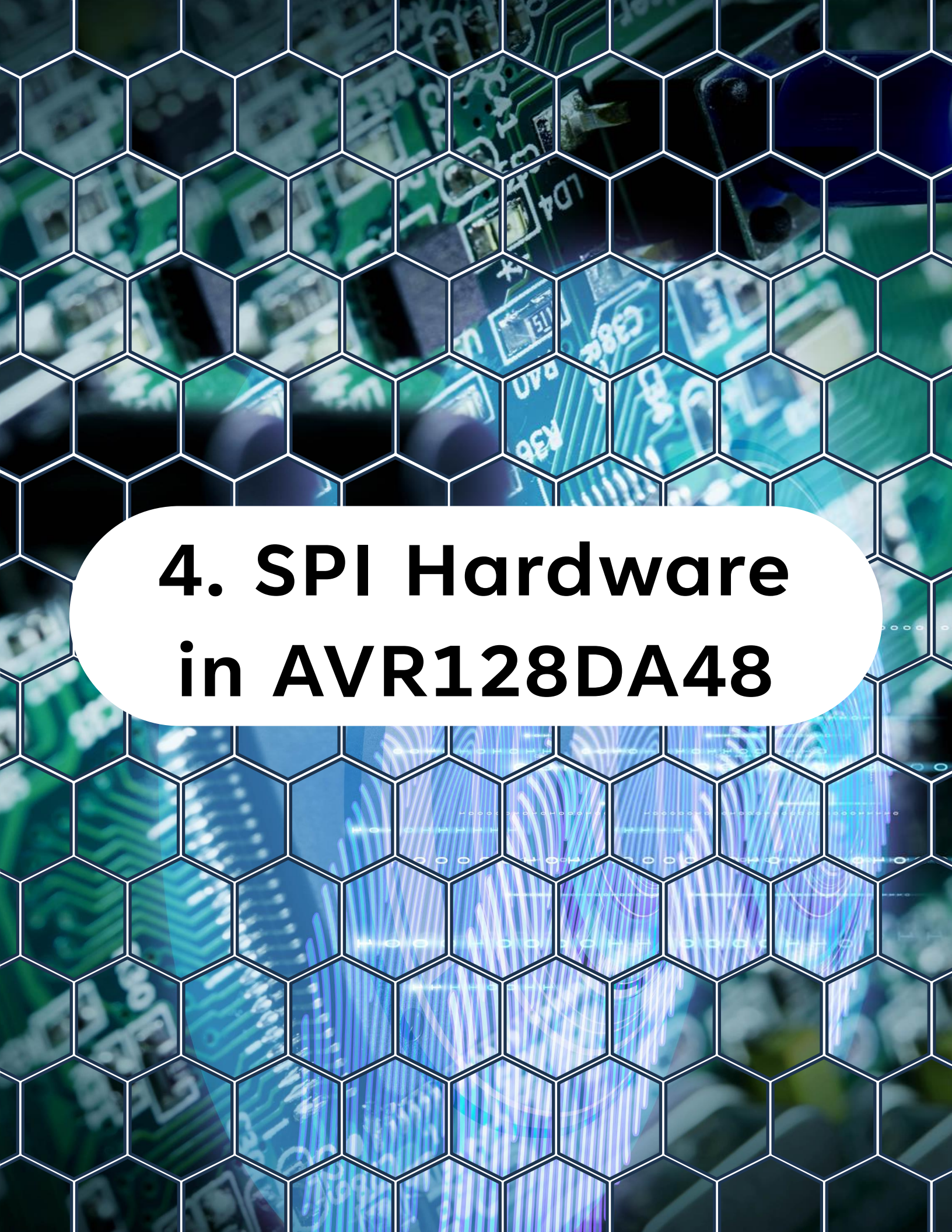
- High speed
- Short distance (on-board communication)
- No framing overhead
- Deterministic timing

USART:

- Longer distance
- Asynchronous
- Framed communication
- PC-friendly

You use SPI for peripherals.

You use USART for terminals and external communication.



4. SPI Hardware in AVR128DA48

4. SPI Hardware in AVR128DA48

The **AVR128DA48** provides **SPI** peripherals that support:

- Master mode
- Slave mode
- Configurable clock polarity (**CPOL**)
- Configurable clock phase (**CPHA**)
- Configurable clock prescaler

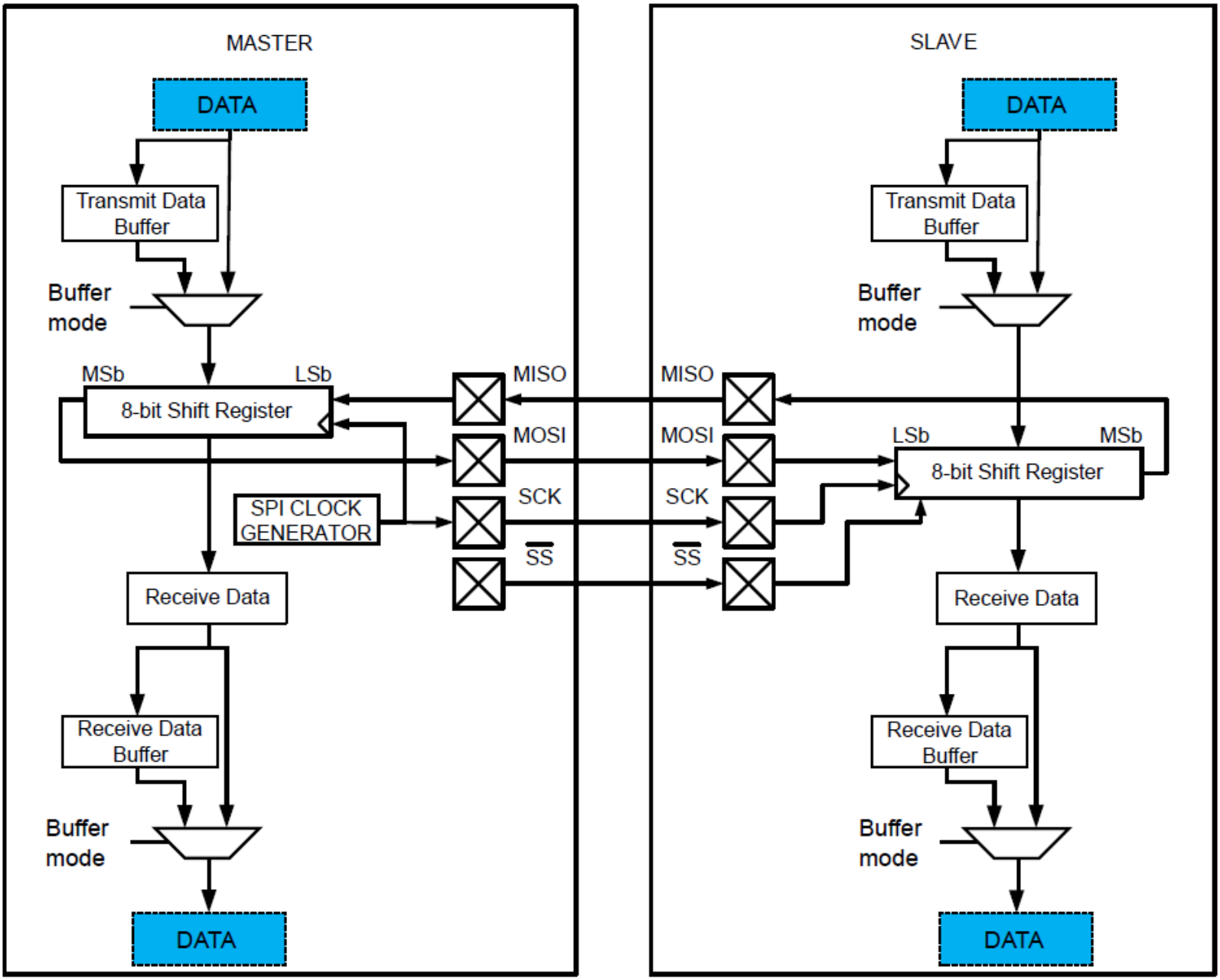
For this lesson, you will use **SPI0 in Master mode**.

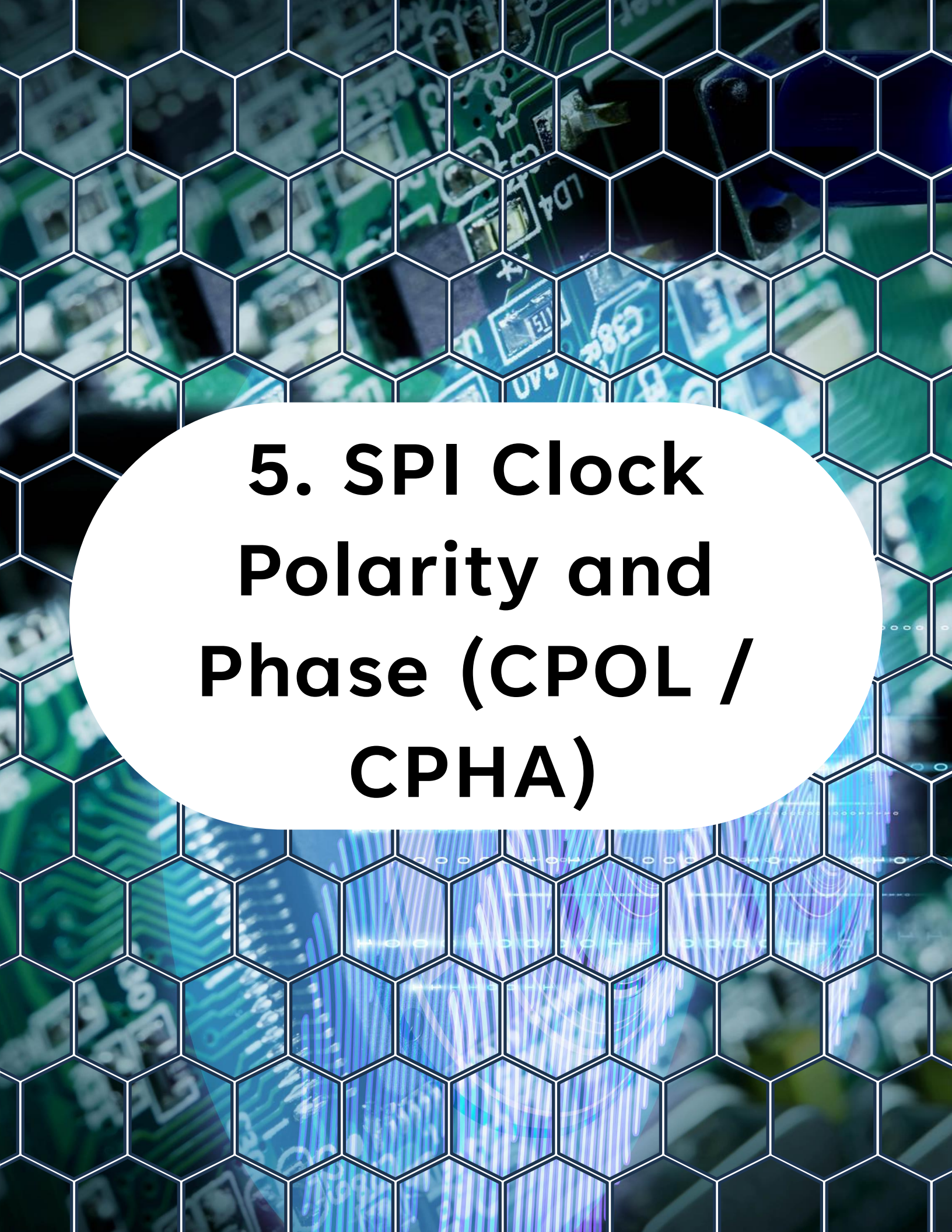
Signals in Master and Slave Mode

Signal	Description	Pin Configuration	
		Master Mode	Slave Mode
MOSI	Master Out Slave In	User defined(1)	Input
MISO	Master In Slave Out	Input	User defined(1,2)
SCK	Serial Clock	User defined(1)	Input
$\overline{SS}$	Slave Select	User defined(1)	Input

4. SPI Hardware in AVR128DA48

SPI Block Diagram





5. SPI Clock Polarity and Phase (CPOL / CPHA)

5. SPI Clock Polarity and Phase (CPOL / CPHA)

There are four SPI modes:

- **Mode 0**: CPOL=0, CPHA=0
- **Mode 1**: CPOL=0, CPHA=1
- **Mode 2**: CPOL=1, CPHA=0
- **Mode 3**: CPOL=1, CPHA=1

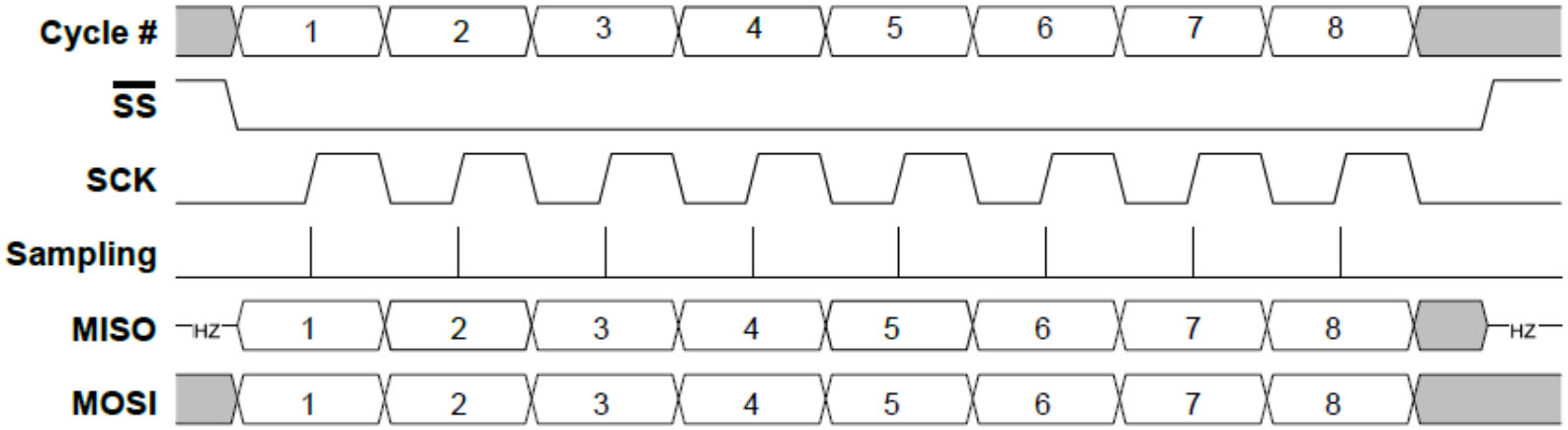
Most devices specify which mode they require.

Mode 0 is common and a good default for learning.

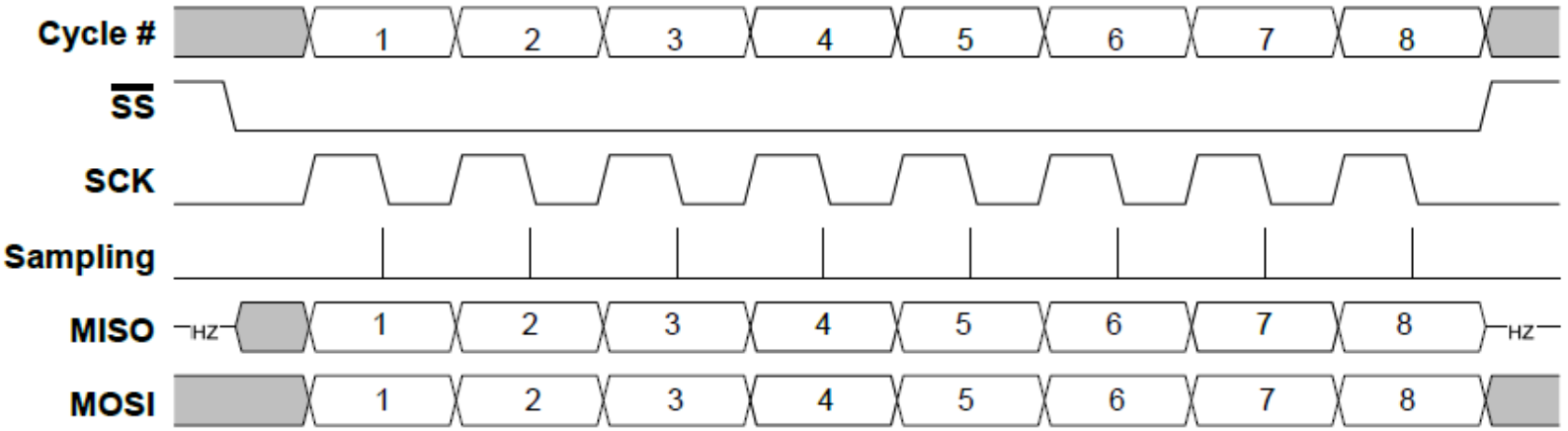
Always check the slave device datasheet.

5. SPI Clock Polarity and Phase (CPOL / CPHA)

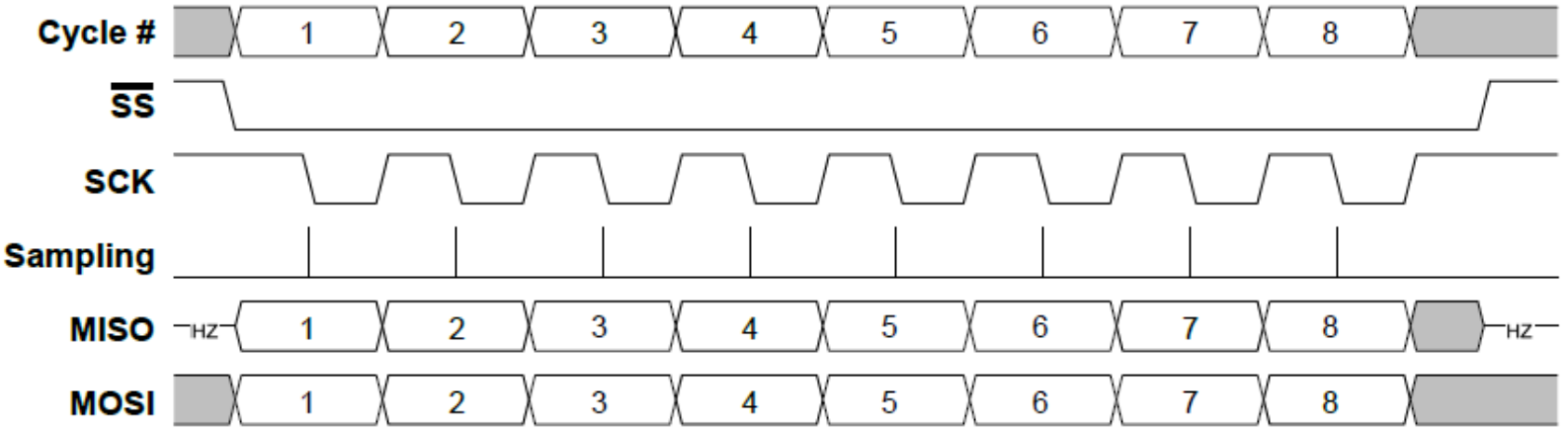
SPI Data Transfer Mode 0



SPI Data Transfer Mode 1

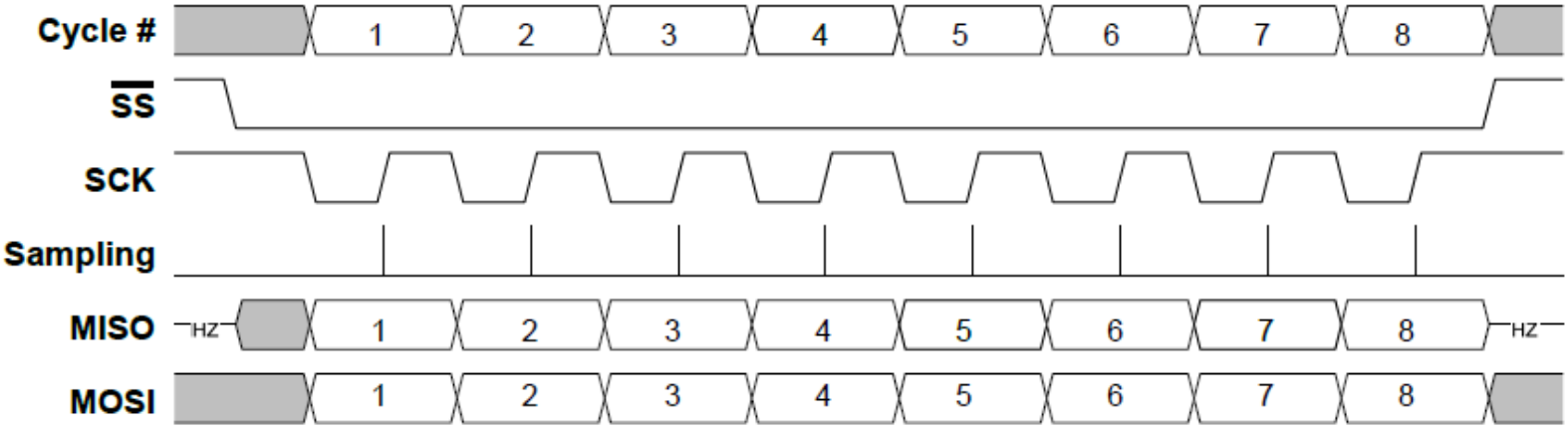


SPI Data Transfer Mode 2



5. SPI Clock Polarity and Phase (CPOL / CPHA)

SPI Data Transfer Mode 3



6. SPI Master Responsibilities

6. SPI Master Responsibilities

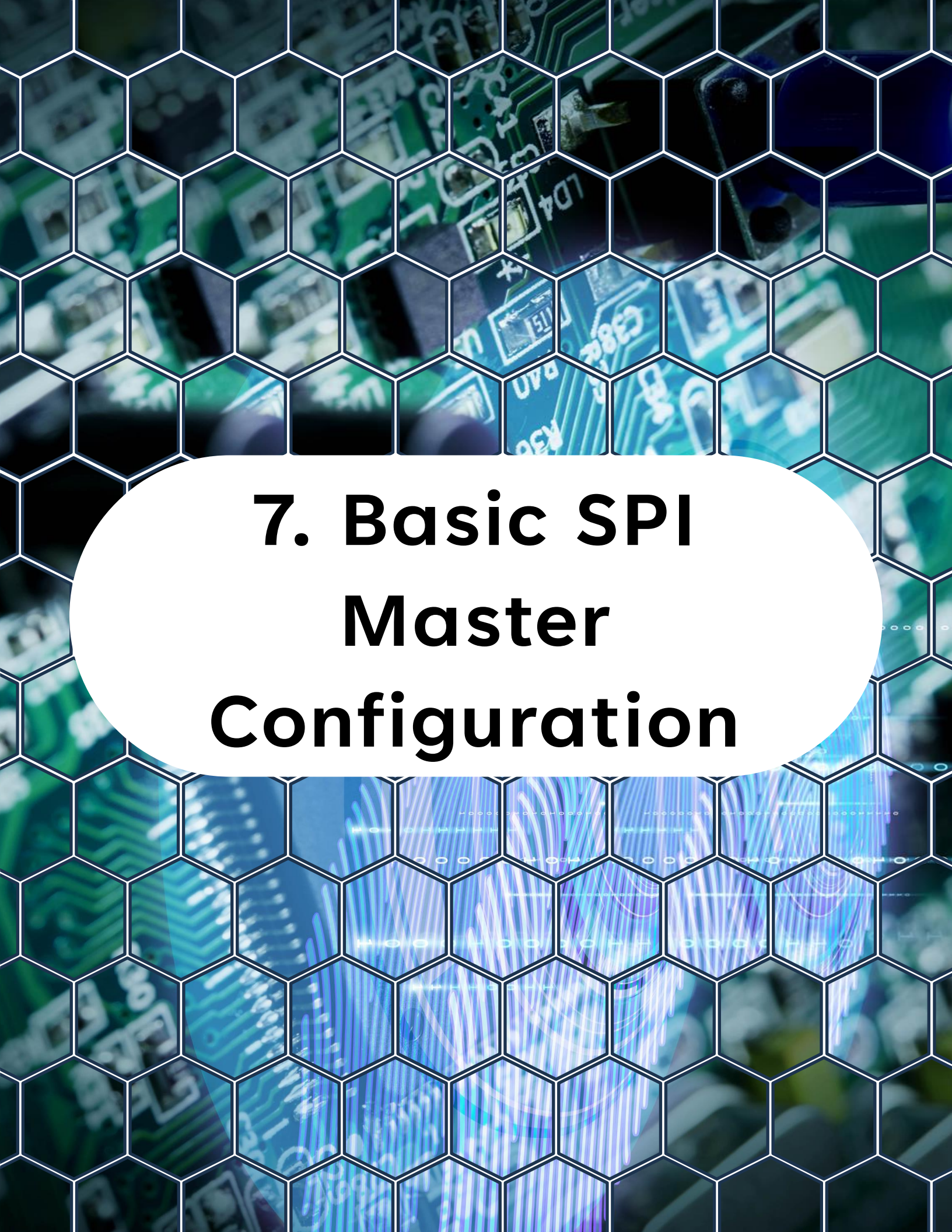
As SPI Master, your MCU must:

- Generate clock
- Drive MOSI
- Control SS (chip select)
- Initiate transfers
- Read received data

Important:

Every SPI transfer is full-duplex.

When you send a byte, you simultaneously receive one.



7. Basic SPI Master Configuration

7. Basic SPI Master Configuration

To configure **SPI0** in Master mode:

- Set MOSI and SCK as outputs
- Set MISO as input
- Configure SPI control registers (**CTRLA**, **CTRLB**, etc)
- Select clock speed
- Enable SPI

When the SPI is configured in Master mode, a write to the **SPIn.DATA** register will start a new transfer.

8. Practical Lab - Basic SPI Master Initialization

8. Practical Lab - Basic SPI Master Initialization

Assumptions

Example pin usage (adjust per board):

- **MOSI** on **PA4**
- **MISO** on **PA5**
- **SCK** on **PA6**
- **SS** on **PA7** (manual control)

Check your board pinout.

Code: SPI0 Initialization (Master Mode)

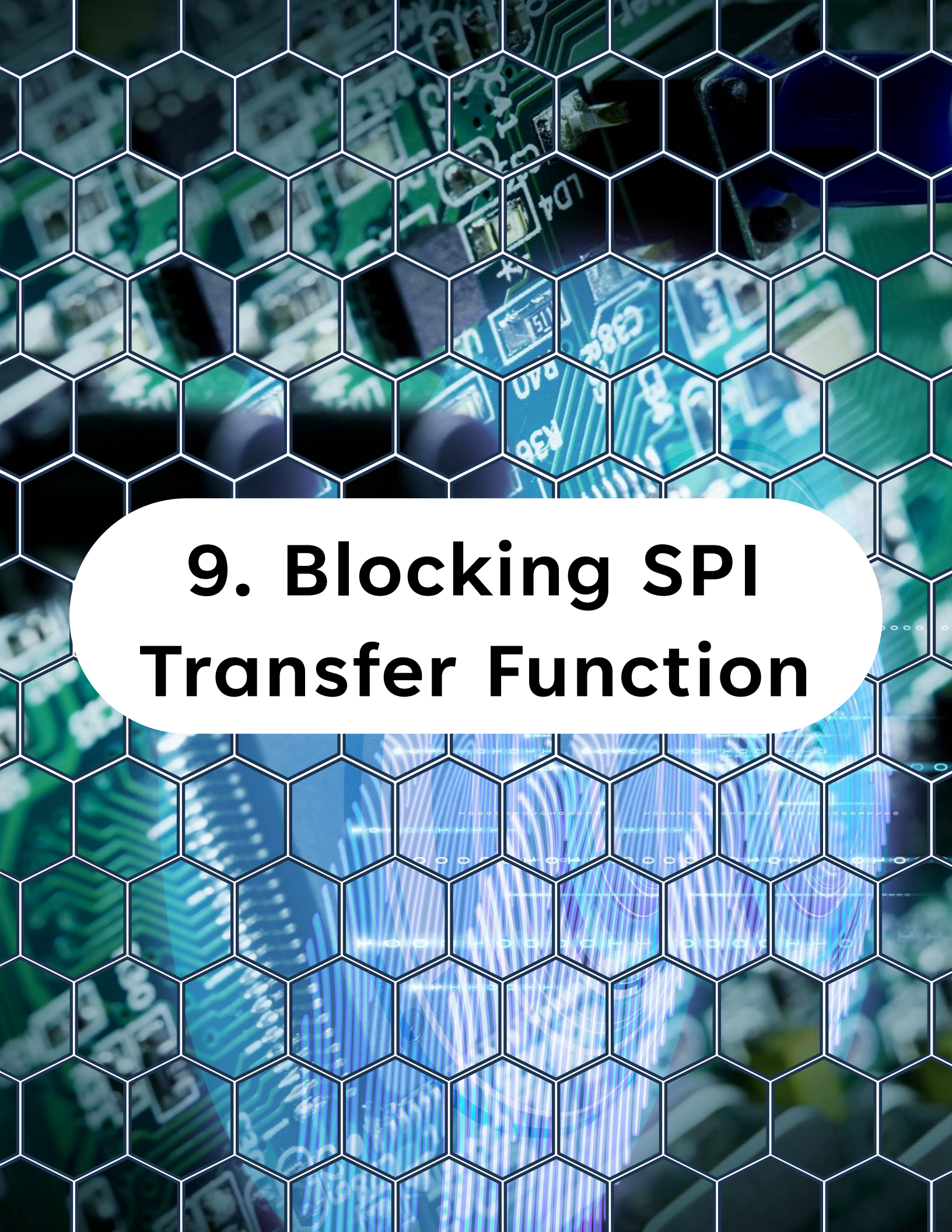
```c
#include <avr/io.h>

static void spi0_init(void)
{
    /* Configure SPI pins */
    /* MOSI, SCK, SS as outputs */
    PORTA.DIRSET = PIN4_bm | PIN6_bm | PIN7_bm;
    /* MISO as input */
    PORTA.DIRCLR = PIN5_bm;

    /* Set SS high (inactive) */
```

8. Practical Lab - Basic SPI Master Initialization

```
11      /* Set SS high (inactive) */
12      PORTA.OUTSET = PIN7_bm;
13
14      /*
15       * Enable SPI in Master mode
16       * Mode 0 (CPOL=0, CPHA=0)
17       * Prescaler = DIV16 (example)
18       */
19      SPI0.CTRLA = SPI_MASTER_bm
20                 | SPI_PRESC_DIV16_gc
21                 | SPI_ENABLE_bm;
22
23      SPI0.CTRLB = 0; /* Mode 0 default */
24  }
```

This sets:

- Master mode
- Clock prescaler
- SPI enabled



9. Blocking SPI Transfer Function

9. Blocking SPI Transfer Function

To transfer a byte:

- Write to **DATA** register
- Wait for interrupt flag (**IF**)
- Read **DATA** register

```c
static uint8_t spi0_transfer(uint8_t data)
{
    SPI0.DATA = data;

    /* Wait until transfer complete */
    while (!(SPI0.INTFLAGS & SPI_IF_bm))
    {
        /* wait */
    }

    /* Clear interrupt flag */
    SPI0.INTFLAGS = SPI_IF_bm;

    return SPI0.DATA;
}
```

Key concept:

- Writing DATA starts the transfer.
- Reading DATA returns the received byte.

10. Chip Select Handling

10. Chip Select Handling

SPI does not automatically manage **SS** for you.

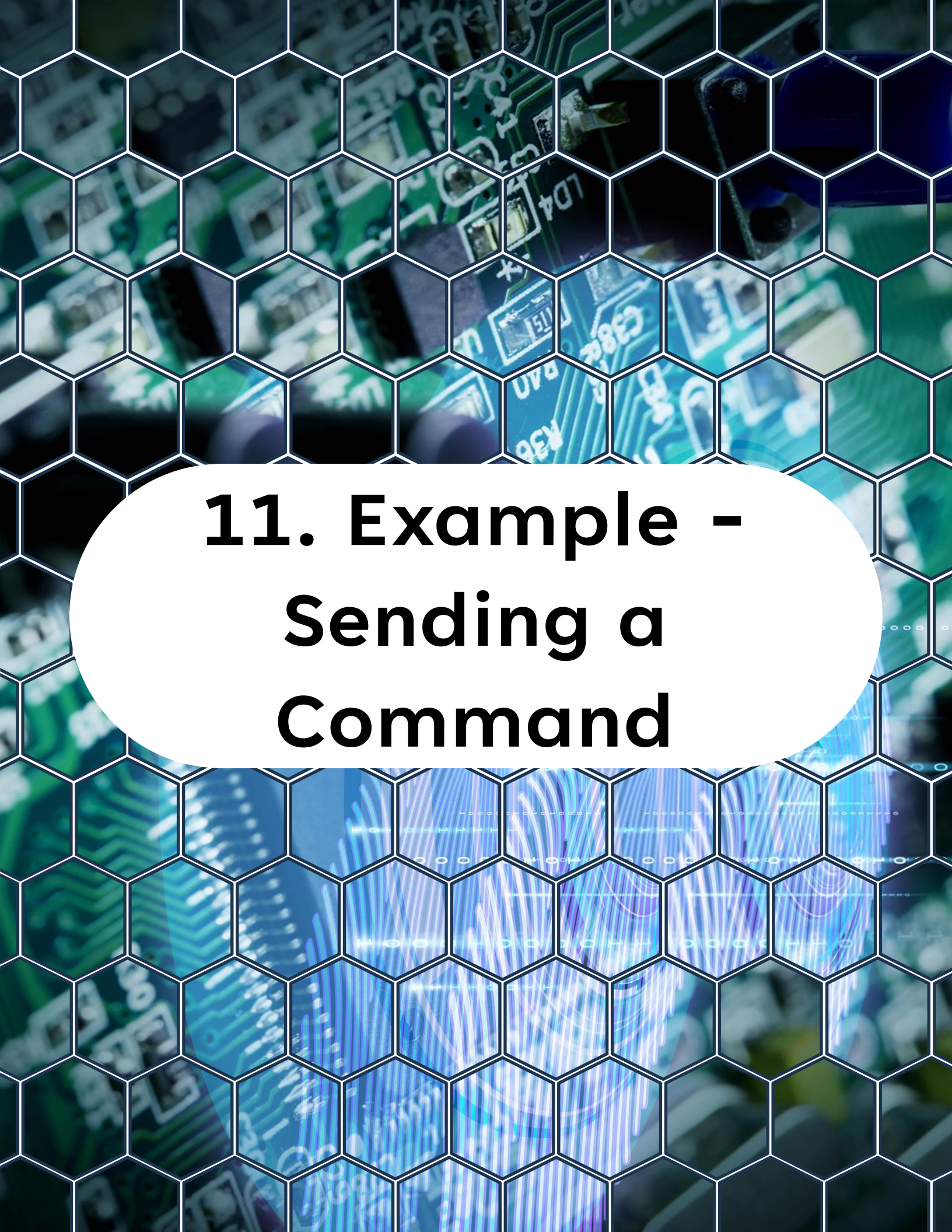
You must:

- Pull **SS** low before transfer
- Transfer bytes
- Pull **SS** high after transfer

Example:

```
// Select the SPI0 device by setting the chip
// select pin low
static void spi0_select(void)
{
    PORTA.OUTCLR = PIN7_bm;
}

// Deselect the SPI0 device by setting the chip
// select pin high
static void spi0_deselect(void)
{
    PORTA.OUTSET = PIN7_bm;
}
```

Never forget this. Many SPI bugs are chip-select errors.



11. Example - Sending a Command

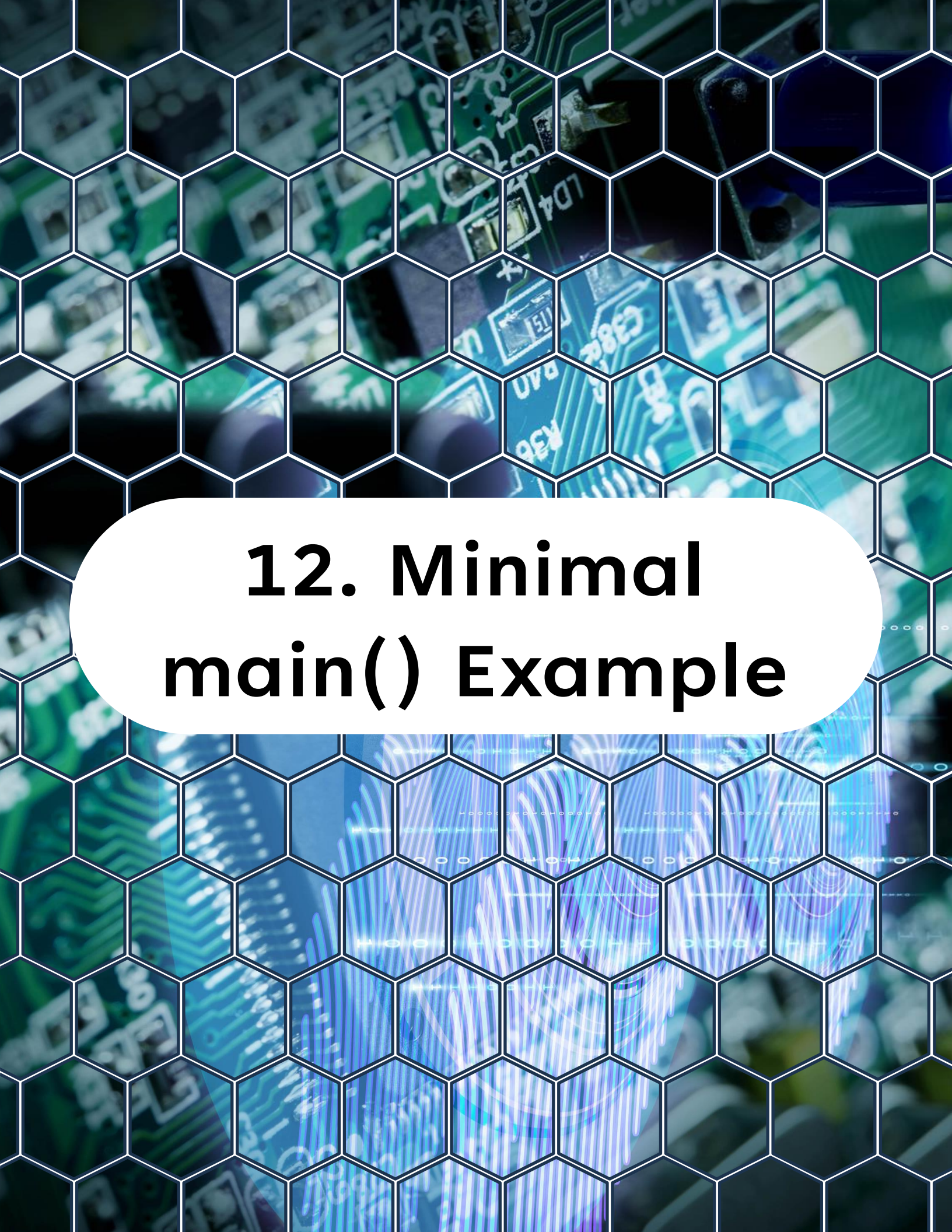
11. Example - Sending a Command

For byte transfer:

```c
static void spi0_send_command(uint8_t cmd)
{
    // Select the SPI device.
    spi0_select();

    // Send the command byte over SPI.
    spi0_transfer(cmd);

    // Deselect the SPI device after sending the command.
    spi0_deselect();
}
```

For multi-byte transfers:

```c
static void spi0_send_buffer(uint8_t *buf, uint8_t len)
{
    // Select the SPI device.
    spi0_select();

    // Send the buffer over SPI.
    for (uint8_t i = 0; i < len; i++)
    {
        spi0_transfer(buf[i]);
    }

    // Deselect the SPI device.
    spi0_deselect();
}
```



12. Minimal main() Example

12. Minimal main() Example

This example:

- Initializes SPI
- Repeatedly sends dummy data
- Useful for scope verification

```c
int main(void)
{
    // Initialize the SPI0 peripheral
    spi0_init();

    while (1)
    {
        // Select the SPI0 device
        spi0_select();

        // Send some data over SPI0
        spi0_transfer(0xAA);
        spi0_transfer(0x55);

        // Deselect the SPI0 device
        spi0_deselect();

        for (volatile uint32_t i = 0; i < 50000; i++)
        {
            /* simple delay */
        }
    }
}
```

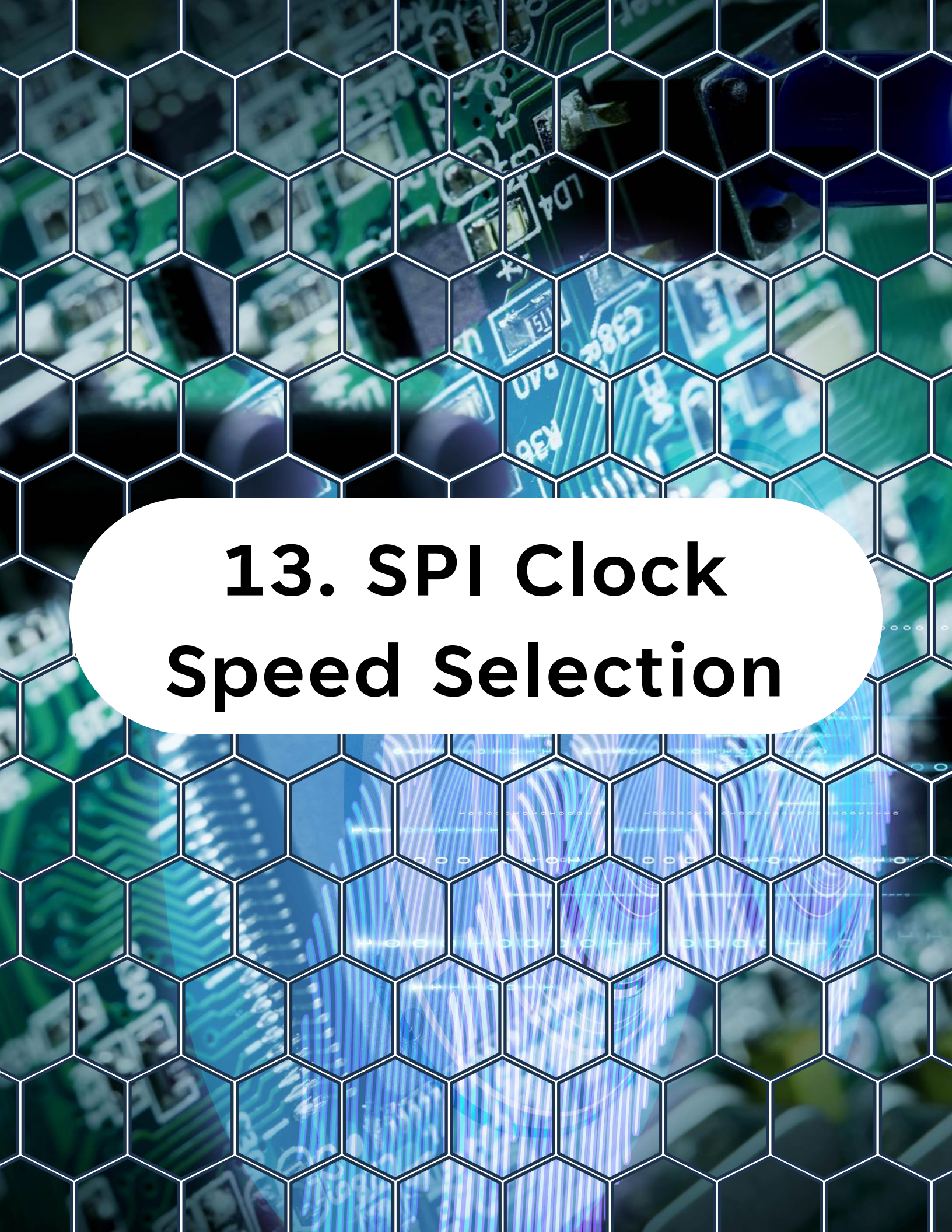
12. Minimal main() Example

Connect scope to:

- SCK
- MOSI
- SS

You should see:

- Clock pulses
- Data shifting
- SS toggling correctly



13. SPI Clock Speed Selection

13. SPI Clock Speed Selection

SPI clock = F_PER / prescaler

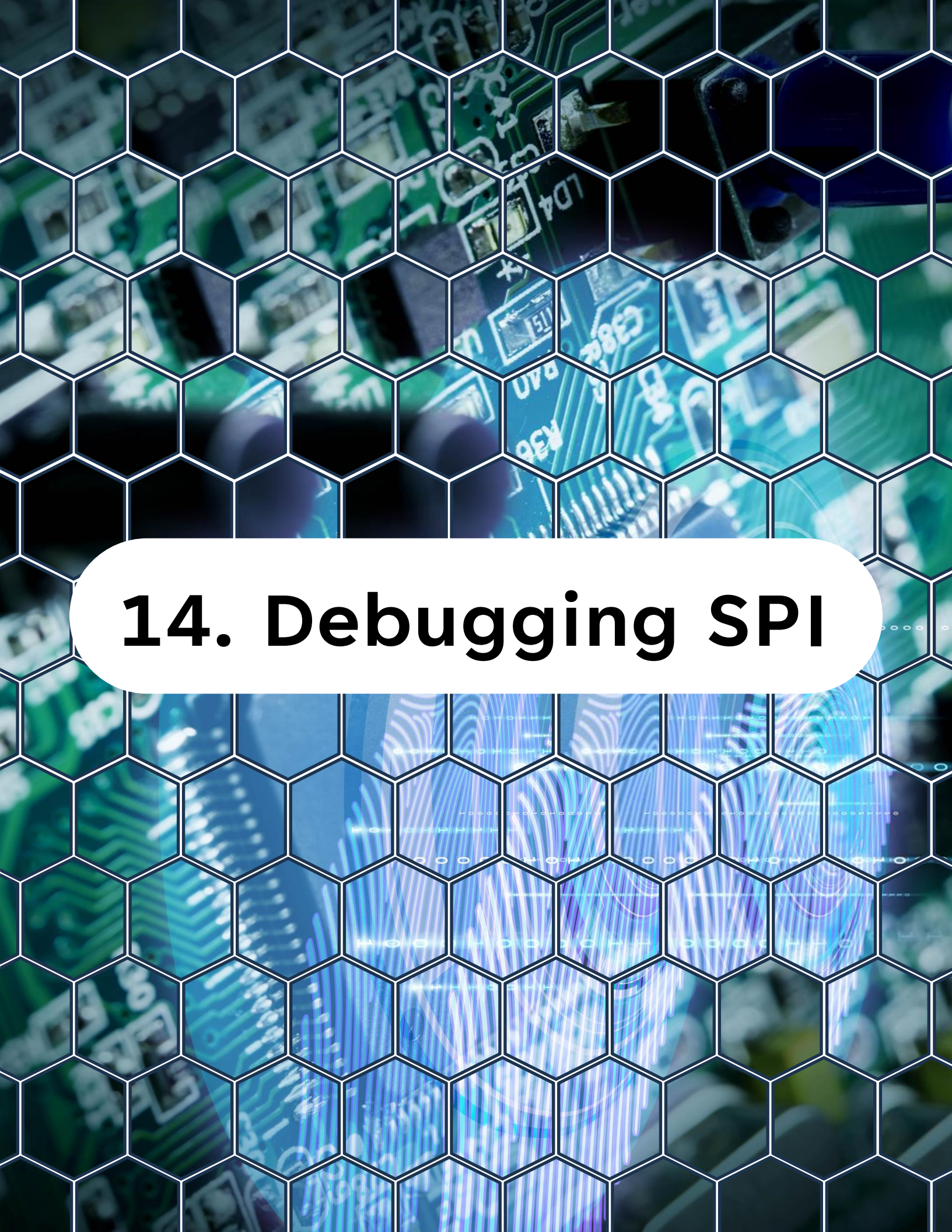
If F_PER = 24 MHz and prescaler = 16:

SPI clock = 1.5 MHz

Always verify:

- Slave maximum SPI clock rating
- Signal integrity on board
- Setup and hold timing

Start slow. Increase speed after validation.



14. Debugging SPI

14. Debugging SPI

If nothing works:

- Check **MOSI**, **MISO**, **SCK** pin directions
- Check **CPOL/CPHA** mode
- Check chip select timing
- Verify SPI clock speed
- Confirm slave powered
- Check wiring

If received data always 0xFF or 0x00:

- MISO not connected
- Slave not selected
- Wrong SPI mode

If transfer never completes:

- SPI not enabled
- Clock not configured
- Interrupt flag not cleared

Always verify signals with a scope or logic analyzer.

15. What You Learned Today

15. What You Learned Today

By the end of Day 13, you understand:

- SPI signal fundamentals
- Master mode configuration
- Clock polarity and phase
- Blocking SPI transfer routine
- Manual chip select handling
- Basic SPI debugging approach

You can now communicate with external hardware devices.

16. What Comes Next

16. What Comes Next

Day 14 - SPI Architecture Patterns

Next you will:

- Build a reusable SPI driver
- Add timeout protection
- Implement non-blocking SPI patterns
- Prepare for real sensor and flash drivers

From here, communication becomes modular and scalable.

"Thanks for watching!

If you enjoyed this video, make sure to hit the like button and subscribe to stay updated with my latest content. Don't forget to check out my other videos for more tips and tutorials on Embedded C, Python, hardware designs, etc. Keep exploring, keep learning, and I'll see you in the next video!"

https://www.linkedin.com/in/yamil-garcia
https://www.youtube.com/@LearningByTutorials
https://github.com/god233012yamil